Prompt do Software e Resposta da I.A.

Table of Contents

Prompt do Software 2
2. Preparando o computador 11
Passo 1: instalar o Python 12
Passo 2: testar se o Python foi instalado 12
Passo 3: instalar o VS Code 12
Passo 4: instalar o pyserial 12
3. Conceitos básicos necessários 13
`print` 13
Variáveis 13
Tipos de dados 14
`if`, `elif` e `else` 14
Funções 15
`try` e `except` 15
4. Primeiro teste sem Arduino 16
5. Criando a primeira lógica de triagem 17
6. Entendendo os dados do Arduino 18
7. Separando texto em dados úteis 19
8. Introdução ao pyserial 21
9. Primeiro teste com Arduino conectado 22
10. Tratando erros 23
Erro de porta COM 24
Dado incompleto 24
Temperatura inválida 25
11. Organizando em funções 25
Função para classificar risco 25
Função para interpretar dados 26
Função para mostrar resultado 27
12. Teste final guiado 27
Checklist de teste sem Arduino 28
Checklist de teste com Arduino 29

13. Melhorias futuras 30
Interface gráfica 30
QR Code 30
Salvamento local 30
Histórico anônimo 31
Integração com áudio 31
Integração com LCD 31
Evolução recomendada do projeto 31

Prompt do Software

PAPEL: Você é um professor de Python, mentor de programação para iniciantes e integrador de sistemas Arduino + Python.

Sua missão é ensinar estudantes do 1º semestre de Ciência da Computação a construir, passo a passo, o software de um robô/totem de triagem hospitalar acadêmico chamado TIAO.

Você **NÃO** deve simplesmente entregar o código pronto. Você deve ensinar o aluno a entender, montar, testar e evoluir o software por etapas.

CONTEXTO DO PROJETO:

Estamos desenvolvendo um protótipo educacional de triagem hospitalar.

O Arduino será responsável por coletar dados físicos dos sensores, como presença, temperatura e, futuramente, batimentos/oxigenação.

O Python será responsável por:

>>> receber os dados enviados pelo Arduino;

>>> interpretar essas informações;

>>> organizar o fluxo da triagem;

>>> exibir mensagens claras para o usuário;

>>> classificar o atendimento de forma preliminar;

>>> gerar uma ficha simples e anônima do atendimento.

O sistema deve funcionar localmente no computador, sem internet, sem nuvem e sem banco de dados externo.

Perfil dos alunos

Considere que os alunos:

- estão no 1º semestre;

- sabem pouco ou nada de Python;

- podem não entender ainda bibliotecas, funções, loops e tratamento de erro;

- precisam aprender fazendo;

- precisam entender o motivo de cada etapa;

- não devem apenas copiar e colar código sem compreender.

OBJETIVO DO PROMPT:

Criar um tutorial completo, didático e progressivo para desenvolver o software do robô TIAO em Python.

O foco deve ser ensinar:

>>> como preparar o ambiente Python;

>>> como instalar bibliotecas;

>>> como testar pequenos trechos antes de integrar tudo;

>>> como receber dados do Arduino;

>>> como validar informações recebidas;

>>> como transformar dados em uma decisão simples de triagem;

>>> como organizar o código em partes pequenas;

>>> como testar o sistema mesmo sem o Arduino conectado;

>>> como evoluir o projeto sem se perder.

Dados esperados do Arduino

O Arduino poderá enviar futuramente uma linha parecida com esta:

"TEMP:36.8;PRESENCA:1;SPO2:98;BPM:82"

Mas, no começo, ensine usando uma versão mais simples:

"TEMP:36.8;PRESENCA:1"

Explique que:

- TEMP representa a temperatura lida pelo sensor;

- PRESENCA indica se há uma pessoa próxima ao totem;

- SPO2 e BPM são dados opcionais para uma fase mais avançada;

- o sistema deve funcionar primeiro com poucos dados e depois evoluir.

TAREFA PRINCIPAL:

Ensine o desenvolvimento do software em Python como um passo a passo prático.

A resposta deve ser organizada como uma trilha de aprendizado, não como um código final pronto.

Divida o tutorial em fases:

>>> Fase 1: entender o papel do Python no projeto;

>>> Fase 2: instalar o Python e preparar o ambiente;

>>> Fase 3: aprender o básico necessário de Python para este projeto;

>>> Fase 4: criar o primeiro programa simples;

>>> Fase 5: aprender a ler dados simulados;

>>> Fase 6: entender o formato dos dados do Arduino;

>>> Fase 7: transformar texto recebido em informações úteis;

>>> Fase 8: criar a lógica simples de triagem;

>>> Fase 9: aprender a usar comunicação serial com pyserial;

>>> Fase 10: integrar Python com Arduino;

>>> Fase 11: tratar erros comuns;

>>> Fase 12: organizar o programa em funções;

>>> Fase 13: testar o sistema completo;

>>> Fase 14: pensar em melhorias futuras.

Regras pedagógicas

- **NÃO** entregar um código completo logo de início.
- **NÃO** resolver tudo sozinho.
- **ENSINAR** o aluno a montar por partes.
- **EXPLICAR** o que cada conceito significa.
- **MOSTRAR** pequenos exemplos isolados.
- **PEDIR** que o aluno teste cada etapa antes de avançar.
- **EVITAR** termos avançados sem explicação.
- **PRIORIZAR** clareza em vez de sofisticação.
- **USAR** exemplos simples e curtos.
- **SEPARAR** teoria, prática e teste.

Conceitos de Python que devem ser ensinados

Explique somente o necessário para o projeto:

- print
- input, se for útil para simular dados
- variáveis
- tipos de dados
- strings
- float e int

- if, elif e else

- loops simples

- funções

- try/except

- importação de bibliotecas

- leitura serial com pyserial

- organização básica do código

Sempre explique cada conceito ligado ao projeto do robô.

Exemplo:

Não explique "if" de forma genérica. Explique que o "if" será usado para decidir se a temperatura indica VERDE, AMARELO ou VERMELHO.

Lógica de triagem inicial

Use uma classificação simples apenas para fins acadêmicos:

- temperatura abaixo de 37.5 graus → VERDE

- temperatura de 37.5 até 38.5 graus → AMARELO

- temperatura acima de 38.5 graus → VERMELHO

ATENÇÃO: deixar claro que isso não é diagnóstico médico.

Privacidade e limites do sistema

O tutorial deve reforçar que o software:

- **NÃO** deve coletar nome completo;
- **NÃO** deve coletar CPF;
- **NÃO** deve armazenar dados sensíveis;
- **NÃO** deve enviar dados para a internet;
- **NÃO** substitui atendimento médico;
- **DEVE** usar ficha anônima ou número de atendimento simples.

FORMATO DA RESPOSTA:

Organize a resposta exatamente neste formato:

1. O que o Python faz no robô

Explique de forma simples o papel do Python no sistema.

2. Preparando o computador

Ensine como instalar Python, VS Code e biblioteca pyserial.

3. Conceitos básicos necessários

Explique apenas os conceitos Python que serão usados no projeto.

4. Primeiro teste sem Arduino

Mostre como simular uma temperatura digitada ou fixa.

5. Criando a primeira lógica de triagem

Ensine como usar if, elif e else para classificar risco.

6. Entendendo os dados do Arduino

Explique o formato "TEMP:36.8;PRESENCA:1" de maneira simples.

7. Separando texto em dados úteis

Ensine como transformar a linha recebida em temperatura e presença.

8. Introdução ao pyserial

Explique o que é comunicação serial, porta COM e baud rate.

9. Primeiro teste com Arduino conectado

Ensine como ler uma linha enviada pelo Arduino.

10. Tratando erros

Explique try/except, erro de porta COM, dado incompleto e Arduino desconectado.

11. Organizando em funções

Mostre como separar o programa em pequenas partes:

- ler dados;
- interpretar dados;
- classificar risco;
- mostrar resultado.

12. Teste final guiado

Crie um checklist de testes para validar o software.

13. Melhorias futuras

Sugira melhorias opcionais:

- interface gráfica;
- QR Code;
- salvamento local;
- histórico anônimo;
- integração com áudio e LCD.

Como apresentar exemplos

Quando mostrar código, siga estas regras:

- mostrar apenas pequenos blocos por etapa;
- explicar antes o objetivo do bloco;
- mostrar o código;
- explicar linha por linha;
- dizer como testar;
- dizer qual resultado esperado;
- só depois avançar.

Estilo da explicação

Use tom de professor paciente e prático.

A explicação deve ser:

- muito didática;

- organizada;

- progressiva;

- fácil para iniciantes;

- focada em prática;

- sem excesso de teoria;

- com exemplos pequenos;

- com alertas de erro comum;

- com checklist no final de cada fase.

SAÍDA ESPERADA:

Ao final, o aluno deve conseguir entender como construir o software por conta própria, etapa por etapa, sem depender de um código pronto.

O objetivo é que o grupo aprenda Python aplicado ao projeto, não apenas copie uma solução.

"Incluir obrigatoriamente a frase: Uso estritamente pedagógico e de apoio."

2. Preparando o computador

Antes de programar, prepare o ambiente.

Você vai precisar de:

- Python instalado;

- VS Code instalado;
- biblioteca `pyserial`.

Passo 1: instalar o Python

Acesse o site oficial do Python e instale a versão mais recente.

Durante a instalação, marque a opção:

```
Add Python to PATH
```

Isso permite usar Python pelo terminal.

Passo 2: testar se o Python foi instalado

Abra o terminal e digite:

```
python --version
```

Resultado esperado:

```
Python 3.x.x
```

Se aparecer erro, provavelmente o Python não foi adicionado ao PATH.

Passo 3: instalar o VS Code

O VS Code será usado para escrever e organizar o código.

Depois de instalar, abra o VS Code e instale a extensão:

```
Python
```

Passo 4: instalar o pyserial

O `pyserial` permite que o Python leia dados enviados pelo Arduino pela porta USB.

No terminal, digite:

```
pip install pyserial
```

Resultado esperado:

```
Successfully installed pyserial
```

Checklist desta fase:

- Python instalado;
- VS Code instalado;
- extensão Python instalada;
- `pyserial` instalado;
- terminal reconhece o comando `python`.

---

3. Conceitos básicos necessários

Você não precisa aprender Python inteiro para começar. Para este projeto, precisa entender apenas o essencial.

`print`

Usado para mostrar mensagens na tela.

Exemplo:

```
print("Bem-vindo ao sistema de triagem do TIAO")
```

Linha por linha:

`print(...)`

Mostra uma mensagem no terminal.

Como testar:

Execute o arquivo Python.

Resultado esperado:

```
Bem-vindo ao sistema de triagem do TIAO
```

Variáveis

Variáveis guardam informações.

Exemplo:

```
temperatura = 36.8
presenca = 1
```

Explicação:

`temperatura = 36.8`

Guarda a temperatura lida.

`presenca = 1`

Guarda se existe alguém perto do robô.

Neste projeto:

```
PRESENCA:1 significa que há alguém próximo.
PRESENCA:0 significa que não há ninguém próximo.
```

Tipos de dados

O Python trabalha com tipos diferentes de informação.

Exemplo:

```
temperatura = 36.8
presenca = 1
mensagem = "Paciente detectado"
```

Aqui temos:

- `36.8`: número decimal, chamado `float`;
- `1`: número inteiro, chamado `int`;
- `"Paciente detectado"`: texto, chamado `string`.

`if`, `elif` e `else`

Esses comandos servem para tomar decisões.

No projeto, eles serão usados para decidir a cor da triagem.

Exemplo:

```
temperatura = 38.0

if temperatura < 37.5:
    print("Classificação: VERDE")
elif temperatura <= 38.5:
    print("Classificação: AMARELO")
else:
    print("Classificação: VERMELHO")
```

Explicação:

```
if temperatura < 37.5:
```

Se a temperatura for menor que 37.5, classifica como verde.

```
elif temperatura <= 38.5:
```

Se não for verde, mas estiver até 38.5, classifica como amarelo.

```
else:
```

Se não caiu em nenhuma condição anterior, classifica como vermelho.

Resultado esperado com `38.0`:

```
Classificação: AMARELO
```

Funções

Funções separam o código em partes menores.

Exemplo:

```
def classificar_temperatura(temperatura):
    if temperatura < 37.5:
        return "VERDE"
    elif temperatura <= 38.5:
        return "AMARELO"
    else:
        return "VERMELHO"
```

Essa função recebe uma temperatura e devolve uma classificação.

Teste:

```
resultado = classificar_temperatura(36.8)
print(resultado)
```

Resultado esperado:

```
VERDE
```

---

try e except

Servem para tratar erros sem quebrar o programa.

Exemplo:

```
texto = "36.8"

try:
    temperatura = float(texto)
    print(temperatura)
except:
    print("Erro ao converter temperatura")
```

Explicação:

```
float(texto)
```

Tenta transformar texto em número decimal.

Se der erro, o programa mostra:

```
Erro ao converter temperatura
```

Isso será útil quando o Arduino enviar algum dado incompleto ou errado.

Checklist desta fase:

- entender `print`;
- entender variáveis;
- entender strings;
- entender `int` e `float`;
- entender `if`, `elif`, `else`;
- entender funções;
- entender `try/except`.

---

4. Primeiro teste sem Arduino

Antes de conectar o Arduino, teste a lógica usando valores simulados.

Objetivo: fazer o Python trabalhar com uma temperatura fixa.

Código:

```
temperatura = 36.8

print("Temperatura recebida:")
print(temperatura)
```

Explicação linha por linha:

```
temperatura = 36.8
```

Cria uma temperatura falsa para teste.

```
print("Temperatura recebida:")
```

Mostra uma mensagem.

```
print(temperatura)
```

Mostra o valor da temperatura.

Como testar:

Salve o arquivo como:

```
triagem_tiao.py
```

Execute no terminal:

```
python triagem_tiao.py
```

Resultado esperado:

```
Temperatura recebida:
36.8
```

Agora teste com `37.8` e depois com `39.0`.

Checklist desta fase:

- criou um arquivo Python;
- executou pelo terminal;
- testou uma temperatura fixa;
- entendeu que ainda não precisa do Arduino.

---

5. Criando a primeira lógica de triagem

Agora vamos transformar a temperatura em uma classificação simples.

A regra acadêmica será:

```
Abaixo de 37.5 → VERDE
De 37.5 até 38.5 → AMARELO
Acima de 38.5 → VERMELHO
```

Atenção: isso **não é diagnóstico médico**.

Objetivo do bloco: classificar uma temperatura simulada.

Código:

```python
temperatura = 38.2

if temperatura < 37.5:
    print("Classificação: VERDE")
elif temperatura <= 38.5:
    print("Classificação: AMARELO")
else:
    print("Classificação: VERMELHO")
```

Explicação:

```python
temperatura = 38.2
```

Define uma temperatura de teste.

```python
if temperatura < 37.5:
```

Verifica se a temperatura está na faixa verde.

```python
elif temperatura <= 38.5:
```

Verifica se está na faixa amarela.

```python
else:
```

Tudo que passar de 38.5 cai aqui.

Como testar:

Troque o valor da temperatura para:

```
temperatura = 36.4
```

Resultado esperado:

```
Classificação: VERDE
```

Depois teste:

```
temperatura = 37.9
```

Resultado esperado:

```
Classificação: AMARELO
```

Depois teste:

```
temperatura = 39.1
```

Resultado esperado:

```
Classificação: VERMELHO
```

Checklist desta fase:

- testou temperatura verde;
- testou temperatura amarela;
- testou temperatura vermelha;
- entendeu que `if` decide o caminho do programa.

---

6. Entendendo os dados do Arduino

O Arduino poderá enviar uma linha assim:

```
TEMP:36.8;PRESENCA:1
```

Isso é uma string, ou seja, um texto.

Ela tem duas partes principais:

```
TEMP:36.8
```

Significa temperatura igual a 36.8 graus.

```
PRESENCA:1
```

Significa que há alguém próximo ao robô.

O ponto e vírgula separa as informações:

```
TEMP:36.8 ; PRESENCA:1
```

Futuramente, o Arduino pode enviar:

`TEMP:36.8;PRESENCA:1;SPO2:98;BPM:82`

Mas no começo use apenas:

`TEMP:36.8;PRESENCA:1`

Porque é melhor fazer pouco funcionando bem do que tentar fazer tudo de uma vez.

Checklist desta fase:

- entendeu que o Arduino envia texto;
- entendeu `TEMP`;
- entendeu `PRESENCA`;
- entendeu o uso de `;`;
- entendeu que BPM e SpO2 ficam para fase avançada.

---

7. Separando texto em dados úteis

Agora vamos pegar este texto:

`TEMP:36.8;PRESENCA:1`

E transformar em dados que o Python consiga usar.

Objetivo: separar temperatura e presença.

Código:

```
linha = "TEMP:36.8;PRESENCA:1"

partes = linha.split(";")

print(partes)
```

Explicação:

`linha = "TEMP:36.8;PRESENCA:1"`

Simula a linha que viria do Arduino.

`partes = linha.split(";")`

Separa o texto onde existe `;`.

`print(partes)`

Mostra o resultado.

Resultado esperado:

```
['TEMP:36.8', 'PRESENCA:1']
```

Agora vamos separar chave e valor.

Código:

```
linha = "TEMP:36.8;PRESENCA:1"

partes = linha.split(";")

temperatura_texto = partes[0].split(":")[1]
presenca_texto = partes[1].split(":")[1]

temperatura = float(temperatura_texto)
presenca = int(presenca_texto)

print(temperatura)
print(presenca)
```

Explicação:

`partes[0]`

Pega a primeira parte:

`TEMP:36.8`

`partes[0].split(":")`

Separa em:

```
TEMP
36.8
```

`[1]`

Pega apenas o valor:

`36.8`

`float(temperatura_texto)`

Transforma `"36.8"` em número decimal.

`int(presenca_texto)`

Transforma `"1"` em número inteiro.

Como testar:

Execute o código.

Resultado esperado:

```
36.8
1
```

Erro comum:

Se o Arduino mandar algo incompleto, como:

```
TEMP:36.8
```

O programa pode dar erro, porque falta `PRESENCA`.

Checklist desta fase:

- usou `split(";");`
- usou `split(":");`
- transformou texto em `float`;
- transformou texto em `int`;
- entendeu que dados incompletos precisam de cuidado.

---

8. Introdução ao pyserial

O `pyserial` é a biblioteca que permite ao Python conversar com o Arduino pela USB.

Quando o Arduino está conectado, o computador cria uma porta.

No Windows, geralmente aparece como:

```
COM3
COM4
COM5
```

No Linux ou Mac, pode aparecer como:

```
/dev/ttyUSB0
/dev/ttyACM0
```

Outro conceito importante é o `baud rate`.

O baud rate é a velocidade de comunicação entre Arduino e Python.

Se no Arduino estiver:

```
Serial.begin(9600);
```

No Python também deve ser:

```
9600
```

Se forem diferentes, os dados podem chegar errados.

Para instalar o pyserial:

```
pip install pyserial
```

Para importar no Python:

```
import serial
```

Explicação:

```
import serial
```

Carrega a biblioteca que permite acessar a porta USB.

Checklist desta fase:

- entendeu o que é comunicação serial;
- entendeu o que é porta COM;
- entendeu o que é baud rate;
- entendeu que Arduino e Python precisam usar a mesma velocidade.

---

9. Primeiro teste com Arduino conectado

Antes do Python ler os dados, o Arduino precisa enviar uma linha pela Serial.

Exemplo simples do que o Arduino deve enviar:

```
TEMP:36.8;PRESENCA:1
```

No Python, o primeiro teste será apenas ler e mostrar a linha.

Objetivo: verificar se o Python consegue receber dados do Arduino.

Código pequeno:

```
import serial

arduino = serial.Serial("COM3", 9600)

linha = arduino.readline().decode().strip()

print(linha)
```

Explicação:

```
import serial
```

Importa a biblioteca de comunicação serial.

```
arduino = serial.Serial("COM3", 9600)
```

Abre a porta `COM3` usando velocidade `9600`.

Atenção: no seu computador pode ser outra porta, como `COM4`.

```
linha = arduino.readline()
```

Lê uma linha enviada pelo Arduino.

```
.decode()
```

Transforma os dados recebidos em texto.

```
.strip()
```

Remove espaços e quebras de linha.

```
print(linha)
```

Mostra a linha recebida.

Como testar:

1. Conecte o Arduino.
2. Veja qual porta COM ele está usando.
3. Troque `"COM3"` pela porta correta.
4. Execute o Python.

Resultado esperado:

```
TEMP:36.8;PRESENCA:1
```

Erro comum:

Se aparecer erro de porta, provavelmente a porta COM está errada ou o Arduino está desconectado.

Checklist desta fase:

- conectou o Arduino;
- confirmou a porta COM;
- usou o mesmo baud rate do Arduino;
- recebeu uma linha no Python;
- imprimiu a linha recebida.

---

10. Tratando erros

Programas reais precisam lidar com problemas.

Alguns erros possíveis:

- Arduino desconectado;
- porta COM errada;
- dado incompleto;
- temperatura inválida;

- presença ausente.

Erro de porta COM

Objetivo: evitar que o programa quebre se a porta estiver errada.

Código:

```
import serial

try:
    arduino = serial.Serial("COM3", 9600)
    print("Arduino conectado")
except:
    print("Erro: não foi possível conectar ao Arduino")
```

Explicação:

`try:`

Tenta executar um código que pode dar erro.

`serial.Serial("COM3", 9600)`

Tenta conectar na porta.

`except:`

Se der erro, entra aqui.

`print("Erro: não foi possível conectar ao Arduino")`

Mostra uma mensagem mais clara.

Dado incompleto

Objetivo: verificar se a linha tem temperatura e presença.

Código:

```
linha = "TEMP:36.8"

if "TEMP:" in linha and "PRESENCA:" in linha:
    print("Dado completo")
else:
    print("Dado incompleto")
```

Resultado esperado:

```
Dado incompleto
```

Explicação:

```
"TEMP:" in linha
```

Verifica se existe temperatura na linha.

```
"PRESENCA:" in linha
```

Verifica se existe presença na linha.

Temperatura inválida

Objetivo: evitar erro quando a temperatura não for número.

Código:

```
temperatura_texto = "abc"

try:
    temperatura = float(temperatura_texto)
    print(temperatura)
except:
    print("Temperatura inválida")
```

Resultado esperado:

```
Temperatura inválida
```

Checklist desta fase:

- tratou erro de conexão;
- verificou dado incompleto;
- tratou temperatura inválida;
- entendeu que erro tratado deixa o sistema mais seguro.

---

11. Organizando em funções

Agora o código começa a ficar maior. Para não virar bagunça, separe em funções pequenas.

Cada função deve ter uma tarefa clara.

Função para classificar risco

Objetivo: receber temperatura e devolver classificação.

Código:

```
def classificar_risco(temperatura):
    if temperatura < 37.5:
        return "VERDE"
```

```
    elif temperatura <= 38.5:
        return "AMARELO"
    else:
        return "VERMELHO"
```

Teste:

```
print(classificar_risco(36.8))
print(classificar_risco(38.0))
print(classificar_risco(39.2))
```

Resultado esperado:

```
VERDE
AMARELO
VERMELHO
```

Função para interpretar dados

Objetivo: transformar texto do Arduino em temperatura e presença.

Código:

```
def interpretar_linha(linha):
    partes = linha.split(";")

    temperatura_texto = partes[0].split(":")[1]
    presenca_texto = partes[1].split(":")[1]

    temperatura = float(temperatura_texto)
    presenca = int(presenca_texto)

    return temperatura, presenca
```

Teste:

```
linha_teste = "TEMP:36.8;PRESENCA:1"

temperatura, presenca = interpretar_linha(linha_teste)

print(temperatura)
print(presenca)
```

Resultado esperado:

```
36.8
1
```

Função para mostrar resultado

Objetivo: exibir a classificação de forma clara.

Código:

```
def mostrar_resultado(temperatura, presenca, risco):
    print("Ficha anônima de atendimento")
    print("Temperatura:", temperatura)
    print("Presença:", presenca)
    print("Classificação:", risco)
```

Teste:

```
mostrar_resultado(36.8, 1, "VERDE")
```

Resultado esperado:

```
Ficha anônima de atendimento
Temperatura: 36.8
Presença: 1
Classificação: VERDE
```

Importante:

A ficha deve ser anônima. Não peça:

- nome completo;
- CPF;
- endereço;
- telefone;
- dados sensíveis.

Use apenas algo simples, como:

```
Atendimento número 001
```

Checklist desta fase:

- criou função para classificar risco;
- criou função para interpretar linha;
- criou função para mostrar resultado;
- entendeu que função evita bagunça;
- manteve a ficha anônima.

---

12. Teste final guiado

Agora o grupo deve testar por partes, sem pular etapas.

Checklist de teste sem Arduino

Teste 1: temperatura verde.

Entrada:

```
TEMP:36.8;PRESENCA:1
```

Resultado esperado:

```
VERDE
```

Teste 2: temperatura amarela.

Entrada:

```
TEMP:38.0;PRESENCA:1
```

Resultado esperado:

```
AMARELO
```

Teste 3: temperatura vermelha.

Entrada:

```
TEMP:39.2;PRESENCA:1
```

Resultado esperado:

```
VERMELHO
```

Teste 4: sem presença.

Entrada:

```
TEMP:36.8;PRESENCA:0
```

Resultado esperado:

```
Nenhuma pessoa detectada
```

Teste 5: dado incompleto.

Entrada:

```
TEMP:36.8
```

Resultado esperado:

```
Dado incompleto
```

Teste 6: temperatura inválida.

Entrada:

```
TEMP:abc;PRESENCA:1
```

Resultado esperado:

```
Temperatura inválida
```

Checklist de teste com Arduino

Antes de testar:

- Arduino conectado;
- porta COM correta;
- baud rate igual no Arduino e no Python;
- monitor serial do Arduino fechado;
- Python executando no terminal.

Durante o teste:

- aproximar a pessoa ou objeto do sensor de presença;
- verificar se `PRESENCA` muda para `1`;
- verificar se a temperatura aparece;
- verificar se a classificação muda conforme a temperatura simulada ou lida.

Resultado esperado geral:

```
Ficha anônima de atendimento
Temperatura: 36.8
Presença: 1
Classificação: VERDE
```

Lembrete importante:

Este projeto é uma simulação acadêmica. A classificação por temperatura é simplificada e **não deve ser usada como decisão médica real**.

Checklist desta fase:

- testou sem Arduino;
- testou com dados simulados;
- testou erros;
- testou Arduino conectado;
- confirmou porta COM;
- confirmou leitura serial;
- confirmou classificação final.

---

13. Melhorias futuras

Depois que o básico estiver funcionando, o grupo pode evoluir o projeto aos poucos.

Interface gráfica

Criar uma tela mais amigável usando bibliotecas como:

`Tkinter`

A interface poderia mostrar:

- mensagem de boas-vindas;
- temperatura;
- presença;
- classificação;
- número de atendimento.

Comece simples. Primeiro terminal, depois interface.

QR Code

A webcam Logitech C270 pode ser usada futuramente para ler QR Code.

O QR Code poderia conter apenas:

`Número de atendimento`

Evite colocar:

- nome;
- CPF;
- dados médicos;
- dados pessoais sensíveis.

Salvamento local

O sistema pode salvar uma ficha anônima em arquivo `.txt` ou `.csv`.

Exemplo de ficha:

```
ATENDIMENTO:001;TEMP:36.8;PRESENCA:1;RISCO:VERDE
```

Não salvar dados sensíveis.

Histórico anônimo

O grupo pode criar um histórico local com os atendimentos simulados.

Exemplo:

```
001 - VERDE - 36.8
002 - AMARELO - 38.0
003 - VERMELHO - 39.2
```

Isso ajuda na apresentação do projeto sem expor pessoas reais.

Integração com áudio

O Python pode futuramente reproduzir mensagens como:

```
Aproxime-se do sensor.
Aguarde a leitura.
Dirija-se ao atendimento indicado.
```

Isso torna o robô mais acessível.

Integração com LCD

O Arduino pode mostrar mensagens em um display LCD, como:

```
Aguarde...
Triagem concluída
Classificação: VERDE
```

O ideal é que:

- Python faça a lógica;
- Arduino controle sensores e display;
- os dois se comuniquem de forma simples.

Evolução recomendada do projeto

Siga esta ordem:

1. Python simulando temperatura;
2. Python classificando risco;
3. Python interpretando texto;
4. Arduino enviando texto fixo;
5. Python lendo Arduino;
6. sensores reais no Arduino;

7. tratamento de erros;
8. ficha anônima;
9. interface;
10. QR Code, áudio e LCD.

Resumo final:

O objetivo não é copiar um código pronto. O objetivo é entender cada parte:

- o Arduino coleta;
- o Python interpreta;
- a lógica classifica;
- o sistema mostra uma ficha anônima;
- tudo funciona localmente;
- nenhuma informação sensível deve ser coletada.

Uso estritamente pedagógico e de apoio.